HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

PROJECT REPORT

Course: Big Data Integration and Processing

Healthcare Big Data Analytics System

GROUP 12

Students:

Tran Duy Khanh	20252266M
Pham Quang Huy	20252271M
Do Hoang Viet	20252744M
Duong Hoang Hai	20252273M
Nguyen Thi Mai Anh	20252265M

Supervisor: Associate Professor Than Quang Khoat

Hanoi, December 18, 2025

Contents

Abstract

This report studies parameter-efficient fine-tuning (PEFT), also referred to as delta-tuning, for adapting large pre-trained language models (PLMs) to downstream tasks. The reference paper by Ding et al. provides a unified view of PEFT methods, explains why tuning only a small subset of parameters can be effective, and reports large-scale empirical results over many NLP benchmarks [1]. In our capstone project, we reproduced and analyzed several representative PEFT methods on the T5-base backbone, focusing on GLUE-SST-2 and GLUE-QNLI. We compared full fine-tuning with LoRA, adapters, prefix tuning, and prompt tuning, and we examined accuracy, trainable parameter ratio, storage cost, and training behavior. Our experiments confirm the main message of the paper: PEFT methods can achieve competitive performance with a tiny fraction of trainable parameters. On QNLI, our best result was prefix tuning with 93.30% accuracy, while prompt tuning, LoRA, and adapter tuning also remained close to full fine-tuning. Overall, this project shows that lightweight adaptation is a practical and scalable alternative to updating all model parameters.

1 Introduction and Motivation

As the healthcare sector is undergoing a throughout digital transformation, the volume of medical data is rapidly increasing in both scale and complexity. The ability to collect, integrate, and analyze such data has become a critical factor in enabling healthcare organizations to improve operational efficiency, optimize resource allocation, and enhance the quality of patient care.

Healthcare data is typically generated from multiple heterogeneous sources, including electronic health records (EHR), insurance data, laboratory results, and medical IoT devices, and so on. This diversity and distribution pose significant challenges for data management and analysis.

To address these challenges, our team conducts the project *"Healthcare Big Data Analytics System"*, which aims to simulate a large-scale data processing system in the healthcare domain.

The system is designed to:

- Collect and integrate data from multiple sources.
- Standardize and store data using a multi-layer Data Lake architecture (Raw → Bronze → Silver → Gold).
- Process data in both batch and streaming modes using Apache Spark.
- Visualize and analyze results using Apache Superset to identify trends, evaluate costs, and predict health risks.

Through this project, the team aims to present a complete healthcare data processing pipeline, from data ingestion – cleaning – storage – analysis – visualization, while also enhancing practical skills in applying Big Data technologies.

1.1 Research Motivation

The healthcare sector is one of the most prominent fields in adopting Big Data and Artificial Intelligence technologies. The analysis of healthcare data is not only academically significant but also highly valuable in practical applications, particularly in healthcare management and public health services.

1.1.1 Applications of Healthcare Data Analytics

Healthcare data analytics provides several important benefits:

- Early detection of health risks and support for physicians in the diagnostic process.
- Prediction of treatment costs and optimization of hospital resources.
- Evaluation of treatment effectiveness based on real-world patient data.

In addition, open healthcare datasets available on platforms such as Kaggle are abundant and well-structured, making them convenient for data collection and processing.

1.2 Research Objectives

- Develop a system model for integrating multi-source healthcare data.
- Build a batch and streaming data processing pipeline based on a Data Lake architecture.
- Conduct experiments using Spark + Kafka + MinIO + MongoDB + Superset deployed on Docker.
- Evaluate system performance in terms of throughput, latency, and reliability.

1.3 Scope and Research Subjects

- Simulated data from public datasets (Patients, Diabetes, Insurance, NoShow).
- No real personal data is used; only simulated data reflecting realistic behaviors.

1.4 Significance and Necessity of the Study

In the context of rapid digital transformation in the healthcare sector, current healthcare information systems still suffer from fragmentation, lack of standardization, and poor interoperability, making data sharing, synchronization, and utilization across institutions challenging. Healthcare data is stored in various formats, including electronic health records (EHR), laboratory results, prescriptions, IoT device data, and insurance records, creating significant challenges for integration and comprehensive analysis.

Therefore, the development of a system capable of integrating and processing multi-source healthcare data in a centralized and unified manner is highly necessary. Such a system enables physicians, hospital administrators, and healthcare authorities to make faster and more accurate decisions based on real-time data. Moreover, it provides a foundation for applying Artificial Intelligence (AI) and Machine Learning (ML) in tasks such as disease prediction, risk detection, cost optimization, and personalized healthcare planning.

1.5 Scientific and Practical Contributions

This project holds strong scientific significance by studying and applying Big Data technologies in the healthcare domain—one of the most data-intensive, diverse, and sensitive sectors. Through the exploration of platforms such as Apache Spark, Kafka, MinIO, MongoDB, and Superset, the team has identified, selected, and integrated modern tools within the Big Data ecosystem to address real-world challenges in healthcare data collection, processing, and analysis.

From a practical perspective, the proposed system can be deployed in hospitals, clinics, or medical research centers to automate data analysis processes and support data-driven decision making. In addition, the project has educational and research value, as students and researchers can directly experiment with a complete Big Data system model, thereby reinforcing theoretical knowledge through hands-on experience.

1.6 Scientific and Practical Contributions

This project holds strong scientific significance by studying and applying Big Data technologies in the healthcare domain—one of the most data-intensive, diverse, and sensitive sectors. Through the exploration of platforms such as Apache Spark, Kafka, MinIO, MongoDB, and Superset, the team has identified, selected, and integrated modern tools within the Big Data ecosystem to address real-world challenges in healthcare data collection, processing, and analysis.

From a practical perspective, the proposed system can be deployed in hospitals, clinics, or medical research centers to automate data analysis processes and support data-driven decision making. In addition, the project has educational and research value, as students and researchers can directly experiment with a complete Big Data system model, thereby reinforcing theoretical knowledge through hands-on experience.

1.7 Summary of Objectives and Expected Outputs

Table 1 summarizes the main objectives, tools used, expected outcomes, and the significance of the project.

Category	**Description**
Overall Objective	Design and simulate a healthcare Big Data system capable of integrating, processing, and visualizing multi-source healthcare data.
Tools Used	Apache Spark, Apache Kafka, MinIO, MongoDB, Apache Superset, Docker.
Expected Outcomes	Develop data processing pipelines for both batch and streaming modes; build a visualization dashboard system for healthcare analytics.
Application Significance	Support hospital management, disease risk prediction, and serve educational and research purposes in Big Data technologies.

Table 1: Summary of project objectives and outcomes

1.8 Chapter Summary

Chapter 1 has presented an overview of the research context, the motivation for selecting the topic, and the necessity of applying Big Data technologies in the healthcare domain. It also outlined the objectives, scope, and both scientific and practical significance of the proposed system.

This chapter highlights that integrating and analyzing multi-source healthcare data is a valuable approach both theoretically and practically. It addresses the issue of fragmented healthcare data, improves hospital management efficiency, and enhances the quality of patient care. The project establishes a foundation for implementing a complete Big Data model in healthcare, covering the entire pipeline from data collection and processing to storage, visualization, and data-driven decision making.

2 Backgrounds

2.1 Big Data Concept

Big Data refers to datasets that are extremely large in volume, generated at high velocity, and diverse in format (commonly described by the 3Vs: Volume, Velocity, Variety).

In the healthcare domain, data may originate from:

- Structured data: electronic health records, patient information, treatment costs.
- Unstructured data: X-ray images, PDF files, MRI images.
- Real-time data: heart rate signals, blood pressure from monitoring devices.

2.2 Architecture and Components of a Big Data System

To effectively process massive volumes of data, a Big Data system typically includes the following core components:

- Data Lake: centralized storage for data in all states.
- ETL (Extract – Transform – Load): processes for extracting, cleaning, and loading data.
- Batch Processing: processing data in batches, suitable for large-scale periodic analysis.
- Stream Processing: real-time data processing for monitoring and rapid response.
- Visualization Layer: supports data analysis and decision-making through visualization.

In this project, Apache Spark is used for both batch and streaming processing, MinIO serves as the Data Lake, Kafka handles data streaming, and Superset is used for visualization.

2.3 Apache Spark

Definition and Role

Apache Spark is a distributed, in-memory data processing framework that supports both batch processing (DataFrame/RDD) and streaming (Structured Streaming). In this project, Spark plays two main roles: (1) batch processing to transform data from Raw $\rightarrow$ Bronze $\rightarrow$ Silver $\rightarrow$ Gold (ETL/ELT); (2) streaming processing (Structured Streaming) to read events from Kafka, perform enrichment/aggregation, and write real-time results to MongoDB or MinIO.

Proposed Deployment Architecture

- Run in standalone cluster mode (for demo) or YARN/Kubernetes for production.
- Structure includes Spark Master and multiple Spark Workers (executors).
- Use spark-submit for batch jobs; use long-running Structured Streaming applications for streaming pipelines.

Standard Configuration (Example)

```
spark.master=spark://spark-master:7077
spark.driver.memory=4g
spark.executor.memory=8g
spark.executor.cores=2
spark.sql.shuffle.partitions=200
spark.hadoop.fs.s3a.endpoint=http://minio:9000
spark.mongodb.output.uri=mongodb://mongo:27017/health.gold
```

Optimization and Operation

- Partitioning: tune spark.sql.shuffle.partitions based on executor cores $\times$ 2.
- Memory management: use off-heap or adjust GC when encountering OOM.
- Checkpointing: required for Structured Streaming to ensure fault tolerance.
- S3A tuning: increase threads and multipart size for large files.
- Speculative execution: consider enabling for batch jobs.

Advantages / Disadvantages

- Advantages: fast in-memory processing, unified API, large ecosystem.
- Disadvantages: requires tuning, high memory cost, cluster management needed.

2.4 Apache Kafka

Definition and Role

Apache Kafka is a distributed streaming platform used for transmitting and managing messages/events with high throughput and retention capability. In this system, Kafka receives healthcare events (e.g., `new_visit`, `lab_result`, `billing_event`) from producers (ingestion scripts, APIs) and provides them to Spark Structured Streaming (consumers) for real-time processing.

Architecture and Key Concepts

- Topic: a logical channel used to store messages (e.g., `health_events`, `lab_results`).
- Partition: each topic is divided into partitions for scalability; the number of partitions determines throughput.
- Producer / Consumer groups: producers send messages; consumer groups process them independently.
- Retention policy: controls how long Kafka stores messages (`retention.ms`) or limits storage size (`retention.bytes`).

Example Configuration (Docker Compose)

```
KAFKA_BROKER_ID: 1
KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT
KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
```

Topic and Partition Design

- Topic `health_events` with partitions = 6 (depending on throughput requirements).
- Use message keys such as `patient_id` to preserve ordering per patient if needed.
- Increase replication factor (≥ 2) in production environments for fault tolerance.

Retention and Compaction

- For important event history, use longer `retention.ms`.
- For changelog-style data (KTable-like), use log compaction (`cleanup.policy=compact`) to retain only the latest record per key.

Optimization and Operation

- Throughput tuning: increase the number of partitions to improve parallelism; ensure the number of consumers does not exceed the number of partitions.
- Monitoring: track consumer lag, throughput, and ISR (in-sync replicas).
- Security: enable TLS and SASL in production environments to protect sensitive healthcare data.

Advantages / Disadvantages

- Advantages: high throughput, strong fault tolerance, temporary storage capability.
- Disadvantages: requires Zookeeper management (in older versions) or complex configuration; scaling introduces additional cost.

2.5 MinIO (Extended)

Definition and Role

MinIO is an S3-compatible object storage system used as the Data Lake layer (Raw/Bronze/Silver/Gold). Its advantages include lightweight deployment, compatibility with the S3 API, and suitability for simulation as well as on-premise deployment.

Storage Structure

- Data is organized into buckets: `raw/`, `bronze/`, `silver/`, and `gold/`.
- Efficient columnar file formats such as Parquet are used to optimize reading performance in Spark.

Important Configuration

- Endpoint: `http://minio:9000`.
- AccessKey/SecretKey: configured in Docker Compose.
- Bucket policy: set read-only permissions for Superset and write permissions for ingestion jobs.

Spark–MinIO Connection (S3A)

- Add the Hadoop-AWS and AWS Java SDK dependencies, or configure S3A for MinIO.
- Example Spark configuration:

```
spark.hadoop.fs.s3a.endpoint = http://minio:9000
spark.hadoop.fs.s3a.access.key = MINIO_ACCESS_KEY
spark.hadoop.fs.s3a.secret.key = MINIO_SECRET_KEY
spark.hadoop.fs.s3a.path.style.access = true
spark.hadoop.fs.s3a.connection.ssl.enabled = false
```

Storage Best Practices

- Use partitioning by year/month/day to improve query efficiency.
- Store Parquet files with compression, such as Snappy, to reduce I/O.
- Avoid creating too many small files, known as the small files problem; aggregate data into appropriately sized files, preferably at least 64 MB.

Example Interaction Using AWS CLI with MinIO

```
mc alias set local http://minio:9000 MINIO_ACCESS MINIO_SECRET
mc mb local/health-raw
mc cp patients.csv local/health-raw/patients/2025-10-01/patients_0001.csv
```

Advantages / Disadvantages

- Advantages: S3-compatible, easy to deploy, and provides good performance for object storage.
- Disadvantages: object storage does not support ACID transactions; an additional layer such as Iceberg or Delta is required if transactions are needed.

2.6 MongoDB

Definition and Role

MongoDB is a NoSQL document-oriented database, suitable for storing streaming results in JSON/BSON format, and serves as the serving layer for real-time dashboards.

Collection Design

- Collection `patients_realtime` stores enriched records (patient_id, last_visit, risk_score, aggregations, ...).
- Indexing: create indexes on `patient_id`, `timestamp`, and `risk_score` for fast queries.

Spark–MongoDB Connection

- Use the MongoDB Spark Connector.
- Example of writing from Spark:

```
result.write \
.format("mongo") \
.option("uri", "mongodb://mongo:27017/health.patients_realtime") \
.mode("append") \
.save()
```

Replication and Durability (Production)

- Configure a replica set (≥ 3 nodes) to ensure fault tolerance and distributed reads.
- Enable authentication and role-based access control for security.

Operational Optimization

- Limit document size; use TTL index if automatic deletion of old data is required.
- Use bulk writes in Spark instead of inserting one record at a time.
- Monitor WiredTiger cache sizing (memory usage).

Advantages / Disadvantages

- Advantages: flexible schema, efficient JSON querying, fast integration with dashboards.
- Disadvantages: not optimized for heavy analytical (OLAP) workloads; requires proper indexing to avoid performance degradation.

2.7 Apache Superset

Definition and Role

Apache Superset is an open-source BI platform used to build dashboards, charts, and perform data exploration. In this system, Superset connects in read-only mode to MongoDB (via MongoDB BI Connector / Presto) or reads Parquet/CSV files from the Gold zone (via Trino, Presto, Dremio, or an intermediate Postgres/ClickHouse).

Data Connectivity

- Direct MongoDB: requires MongoDB BI Connector to expose MySQL wire protocol, or use Trino/Presto connector for MongoDB.
- Via Parquet: Superset typically connects to SQL engines (Postgres, Trino, ClickHouse). When using MinIO/Parquet, a query engine (Trino/Presto) should be placed between Superset and MinIO.

Setup Workflow (Suggested)

1. Install Trino with Hive/MinIO (S3) connector to expose Parquet as tables.
2. Superset connects to Trino via SQLAlchemy URI to query Gold tables.
3. For MongoDB (real-time use), configure MongoDB BI or replicate summarized data into Postgres/ClickHouse for Superset.

Dashboard Design

- Basic charts: bar chart, line chart, heatmap; apply filters such as date range, group by age/gender.
- Real-time charts: Superset supports refresh intervals to display near real-time data (depending on backend support).

Security and Access Control

- Use role-based access control (RBAC) in Superset.
- Configure database connections as read-only with separate credentials.

Advantages / Disadvantages

- Advantages: powerful UI, rich chart support, large community.
- Disadvantages: not natively real-time (depends on backend); requires a SQL engine layer for querying Parquet/object storage.

2.8 Docker (Extended)

Definition and Role

Docker Compose is used to simulate a multi-service environment, enabling rapid deployment of Spark Master/Worker, Kafka, Zookeeper, MinIO, MongoDB, and Superset within a single host/VM for experimentation and demonstration.

Example docker-compose.yml (Summary)

```yaml
version: '3.8'
services:
  zookeeper:
    image: wurstmeister/zookeeper
    ports: ['2181:2181']
  kafka:
    image: wurstmeister/kafka
    environment:
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
    ports: ['9092:9092']
  minio:
    image: minio/minio
    command: server /data
    environment:
      MINIO_ROOT_USER: minioadmin
      MINIO_ROOT_PASSWORD: minioadmin
    ports: ['9000:9000']
  mongo:
    image: mongo:6.0
    ports: ['27017:27017']
  spark-master:
    image: bitnami/spark:latest
    environment:
      - SPARK_MODE=master
    ports: ['7077:7077','8080:8080']
  spark-worker:
    image: bitnami/spark:latest
    environment:
      - SPARK_MODE=worker
      - SPARK_MASTER_URL=spark://spark-master:7077
  superset:
    image: apache/superset
    ports: ['8088:8088']
```

Docker Compose Configuration Notes

- Network: place all services in the same network to allow hostname-based access (e.g., kafka:9092, minio:9000).
- Volumes: map volumes for MongoDB and MinIO to persist data.
- Resource limits: restrict CPU/Memory usage for constrained environments.

- Healthchecks: define healthchecks to ensure service readiness.

Startup and Testing

```
docker-compose up -d
docker-compose logs -f spark-worker
docker exec -it kafka kafka-topics.sh --create \
--topic health_events \
--bootstrap-server kafka:9092 \
--partitions 6 \
--replication-factor 1
```

Advantages / Disadvantages

- Advantages: fast deployment, reproducible environments, suitable for development/testing.
- Disadvantages: not a replacement for production orchestration (e.g., Kubernetes); performance differs from dedicated VM deployments.

2.9 Integrated Workflow of System Components

1. Ingestion: Python scripts read CSV files and send events to Kafka or upload files to MinIO (Raw layer).
2. Batch processing (Spark): Spark reads Parquet/CSV from MinIO (Raw), transforms data, and writes Parquet to Bronze/Silver/Gold layers.
3. Streaming processing (Spark Structured Streaming): Spark reads from Kafka topic `health_events`, performs enrichment (e.g., join with Silver data by `patient_id`), and writes results to MongoDB (real-time) and/or appends Parquet to Gold.
4. Serving/BI (Superset): Superset connects to Trino/Presto or MongoDB BI to visualize real-time dashboards and historical reports.

2.10 Chapter Summary

Chapter 2 presented the theoretical foundations and core technologies used in the system, including Apache Spark, Apache Kafka, MinIO, MongoDB, Apache Superset, and Docker. Each technology was analyzed in terms of its characteristics, roles, operational principles, and integration within the overall architecture of the healthcare data integration and analytics system.

This theoretical foundation provides a solid technical basis for subsequent chapters, where the system will be designed, implemented, tested, and evaluated in a simulated real-world environment.

3 System Design

3.1 Overall Objectives

Build a complete Big Data system capable of:

- Integrating healthcare data from multiple sources.
- Storing data using a Data Lake architecture.
- Processing data with Apache Spark (batch & streaming).
- Analyzing, visualizing, and evaluating healthcare trends.

3.2 Specific Objectives

- Collect and integrate healthcare data from multiple sources (at least three datasets), including patient information, insurance data, and laboratory test results.
- Standardize and store data using a multi-layer Data Lake architecture (Raw → Bronze → Silver → Gold) on a distributed storage platform (MinIO).
- Build data processing and analytics pipelines using Apache Spark, supporting both batch and streaming processing.
- Perform visualization and analysis using Apache Superset to detect trends, evaluate costs, and predict disease risks.

Through this system, the project aims to demonstrate a complete Big Data pipeline in healthcare, from data ingestion – cleaning – storage – processing – analysis, thereby enhancing practical understanding of modern Big Data technologies.

3.3 System Requirements

Technical Requirements:

- Services are deployed using Docker Compose.
- Integration of MinIO, MongoDB, Spark, Kafka, and Superset.

Data Requirements:

- Data formats: CSV or JSON.
- Includes patient information, biometrics, costs, and laboratory results.

3.4 Data Processing Workflow

The system is designed based on a **Data Lake – Streaming Hybrid Architecture**, combining:

- Batch Processing
- Streaming Processing

This architecture supports both long-term storage and real-time responsiveness, which is suitable for healthcare data characteristics.

3.4.1 (1) Raw Data Ingestion (Raw Zone)

- Raw data is collected from multiple sources such as CSV files, JSON files, or simulated healthcare APIs.
- All raw data is stored centrally in MinIO (Raw Zone).
- This layer preserves original data for validation, reprocessing, and recovery.

3.4.2 (2) Batch Processing

- Apache Spark performs periodic batch processing.
- Processing stages include:
  - **Bronze Zone**: Basic cleaning, removing invalid or duplicate records.
  - **Silver Zone**: Data standardization, schema alignment, and joins across datasets (patients, insurance, diagnosis).
  - **Gold Zone**: Aggregated data ready for analysis (age groups, gender, disease types, treatment costs, risk indicators).
- Gold Zone outputs are stored in Parquet or CSV for BI tools such as Superset.

3.4.3 (3) Streaming Processing

- Apache Kafka handles real-time event ingestion (e.g., new visits, lab results, payment events).
- Spark Structured Streaming consumes Kafka data in micro-batches.
- Data is processed, enriched with Silver Zone data, and written to MongoDB.
- Spark checkpointing ensures **exactly-once processing**.

3.4.4 (4) Analytics and Serving Layer

- Users access analytics via Apache Superset.
- Superset connects to:
  - MongoDB for real-time dashboards
  - Gold Zone (MinIO) for historical and aggregated analysis
- Dashboards support filtering by time, patient groups, diseases, and regions.

3.5 Advantages of the Proposed Architecture

- **Scalability**: Spark, Kafka, and MinIO support horizontal scaling.
- **Flexibility**: Easy integration of new data sources and analytics tools.
- **Fault Tolerance**: Automatic recovery mechanisms in Kafka and Spark.
- **Real-time Insights**: Continuous data updates enable timely decision-making.
- **Low Cost**: Built entirely on open-source technologies.

3.6 Chapter Summary

This chapter presented the overall system design and implementation technologies for healthcare data integration and analytics. The four-layer architecture (deployment, storage, processing, analytics) ensures scalability, performance, and flexibility. By combining Data Lake architecture with both batch and streaming processing, the system supports long-term storage and real-time analytics simultaneously. These foundations enable the next chapter to focus on system implementation, data flow, experimental setup, and performance evaluation in a Docker-based environment.

4 System Development

4.1 System Architecture

The system is designed based on a **Big Data Architecture**, consisting of four main functional layers: the **Deployment Layer**, **Storage Layer**, **Processing Layer**, and **Analytics Layer**. This architecture ensures flexibility, scalability, and ease of management during both deployment and operation.

The four-layer structure of the system is described as follows:

4.1.1 Deployment Layer

All system components are deployed using **Docker Compose**, ensuring environment reproducibility and flexible scalability. Docker Compose allows multiple containers (Spark, Kafka, MongoDB, MinIO, Superset) to be defined and launched within a shared internal network, simplifying setup and minimizing configuration conflicts.

This layer serves as the operational and monitoring foundation of the system.

4.1.2 Storage Layer

This layer includes **MinIO** and **MongoDB**:

- **MinIO** acts as a Data Lake, storing both raw data (Raw Zone) and processed data (Bronze, Silver, Gold). It uses an S3-compatible protocol, enabling efficient data read-/write operations with Spark and supporting scalable storage.
- **MongoDB** functions as a real-time database, storing continuously updated results from Spark Streaming, which are used for real-time visualization in Superset.

4.1.3 Processing Layer

This layer consists of two main mechanisms: **Batch Processing** and **Streaming Processing**, orchestrated by Apache Spark and Apache Kafka:

- **Batch Processing** uses Spark to process large volumes of data periodically, applying the Extract–Transform–Load (ETL) pipeline to clean, standardize, and aggregate data.
- **Streaming Processing** uses Kafka as a message broker to handle real-time, event-driven data. Spark Structured Streaming continuously reads data from Kafka topics, processes it, and writes the results to MongoDB.

The combination of these two mechanisms (Batch + Streaming) enables real-time analytics while maintaining the consistency and integrity of aggregated data.

4.1.4 Analytics Layer

This layer uses **Apache Superset** for data visualization from MinIO and MongoDB. Superset allows users to create dashboards, charts, and interactive reports for management, research, and decision-making purposes.

Visualizations can display real-time data (when connected to MongoDB) or historical data (when querying the Gold Zone in MinIO via Presto/Trino).

4.2 Data Flow Description

The data flow is organized into five main steps, ensuring a sequential processing pipeline and optimized system performance.

Step 1 – Data Ingestion

- Data is collected from four healthcare datasets: *Patients*, *Insurance*, *Diabetes*, and *NoShows*.
- CSV/JSON files are downloaded and uploaded to **MinIO (Raw Zone)** using Python scripts.
- In parallel, **Kafka** simulates real-time data streams (e.g., new patient visits or laboratory test results), which are sent to **Spark Streaming** for processing.

Step 2 – Data Cleaning and Standardization (Bronze → Silver)

- **Apache Spark** (batch mode) reads data from the Raw Zone, removes missing values, corrects formatting errors, and standardizes data types (numeric, string, date).
- The cleaned data is written to the **Bronze Zone**.
- In the next stage, Spark transforms the data into the **Silver Zone**, where datasets (patients, insurance, diabetes) are integrated by joining on common keys such as `patient_id` or `age + gender`.

Column Name Standardization (snake_case) Column names are standardized into `snake_case` format to avoid errors in SQL queries and dashboard generation.

```
from pyspark.sql.functions import col

rename_map = {
    "Blood Type": "blood_type",
    "Date of Admission": "date_of_admission",
    "Medical Condition": "medical_condition",
    "Billing Amount": "billing_amount",
    "Admission Type": "admission_type",
    "Discharge Date": "discharge_date",
    "Test Results": "test_results",
    "Hipertension": "hypertension",
    "Handcap": "handicap",
    "Date.diff": "date_diff",
    "PatientId": "patient_id"
}

for old_col, new_col in rename_map.items():
    if old_col in df.columns:
        df = df.withColumnRenamed(old_col, new_col)
```

Name Casing Normalization Patient names are normalized into title case to reduce duplicated records caused by inconsistent capitalization.

```
from pyspark.sql.functions import initcap, trim

if "name" in df.columns:
    df = df.withColumn("name_norm", initcap(trim(col("name"))))
```

Zero-as-Null Flag For the Diabetes dataset, some fields use the value `0` to represent missing values. Instead of removing the original value, additional flag columns are created.

```
from pyspark.sql.functions import when

df = df.withColumn(
    "insulin_is_missing",
    when(col("Insulin") == 0, True).otherwise(False)
)

df = df.withColumn(
    "blood_pressure_is_missing",
    when(col("BloodPressure") == 0, True).otherwise(False)
)

df = df.withColumn(
    "glucose_is_missing",
```

```
    when(col("Glucose") == 0, True).otherwise(False)
)
```

Quarantine Zone Rows that fail validation should not be silently dropped. Instead, they are written to a separate quarantine table with a rejection reason.

```
from pyspark.sql.functions import lit

invalid_df = df.filter(
    (col("age") < 0) |
    (col("age") > 115)
).withColumn(
    "reject_reason",
    lit("Invalid age value")
)

valid_df = df.filter(
    (col("age") >= 0) &
    (col("age") <= 115)
)

invalid_df.write.mode("append").parquet(
    "s3a://silver-healthcare/bronze_quarantine/"
)
```

Deduplication by Natural Key Instead of applying `dropDuplicates()` to all columns, duplicates are removed using meaningful natural keys for each dataset.

```
natural_keys = {
    "patients": ["name_norm", "date_of_admission", "hospital"],
    "noshows": ["patient_id", "appointment_id"],
    "diabetes": ["id"],
    "insurance": ["age", "sex", "bmi", "region"]
}

dataset_name = "patients"

if dataset_name in natural_keys:
    df = df.dropDuplicates(natural_keys[dataset_name])
```

Step 3 – Data Integration and Aggregation (Silver → Gold)

- **Apache Spark** performs statistical aggregation to compute:
  - Average treatment cost by age group or disease category.
  - Correlations between healthcare indicators (e.g., BMI, Glucose) and insurance costs.

- The resulting data is written to the **Gold Zone**, where it is ready for visualization and reporting.

Step 4 – Real-time Streaming Processing

- **Apache Kafka** continuously ingests new data streams (events).
- **Spark Structured Streaming** reads data from Kafka, processes it, and writes the results to **MongoDB**.
- **MongoDB** acts as a real-time data store, supporting live dashboards and instant data access.

Step 5 – Analytics and Visualization

- **Apache Superset** connects to both the **Gold Zone (MinIO)** and **MongoDB**, enabling:
  - Real-time dashboard creation
  - Analysis of healthcare costs, patient distributions, and disease trends

References

[1] Ding, N., Qin, Y., Yang, G., et al.: Parameter-efficient fine-tuning of large-scale pre-trained language models. Nature Machine Intelligence **5**, 220–235 (2023)